Linear Regression, Gradient Descent & Regularization

A Complete Zero-to-Pro Tutorial with scikit-learn

From plain LinearRegression() to SGDRegressor, Momentum, RMSprop,
L1/L2 Regularization, and Real GridSearchCV Tuning

01 Part 1 — The Basics: Ordinary Least Squares LinearRegression

02 Part 2 — Why Gradient Descent? Normal Equation vs. Iterative Learning

03 Part 3 — SGDRegressor: Gradient Descent + L1/L2/ElasticNet Regularization

04 Part 4 — Advanced: Momentum & RMSprop from Scratch

05 Part 5 — Pro Tips: 6 Mistakes Beginners Make (and the fixes)

06 Part 6 — The Ultimate Beginner Template (full code)

07 Part 7 — Cheat Sheets & Quick Reference

PART 1 · THE BASICS

What Is Linear Regression?

Linear Regression is the simplest way to teach a computer to draw the best straight line through a cloud of data points, so that it can predict a numeric value (like a price, a score, or a temperature) from one or more input features. In its simplest one-feature form, the line is:

$$y = w \cdot x + b$$

- `y` — the value we are trying to predict (the target)
- `x` — the input feature (e.g. house size)
- `w` — the weight / slope (how much y changes per unit of x)
- `b` — the bias / intercept (the value of y when x = 0)

Training the model simply means finding the values of w and b that make the line fit the data as closely as possible — that is, that make the total prediction error as small as possible.

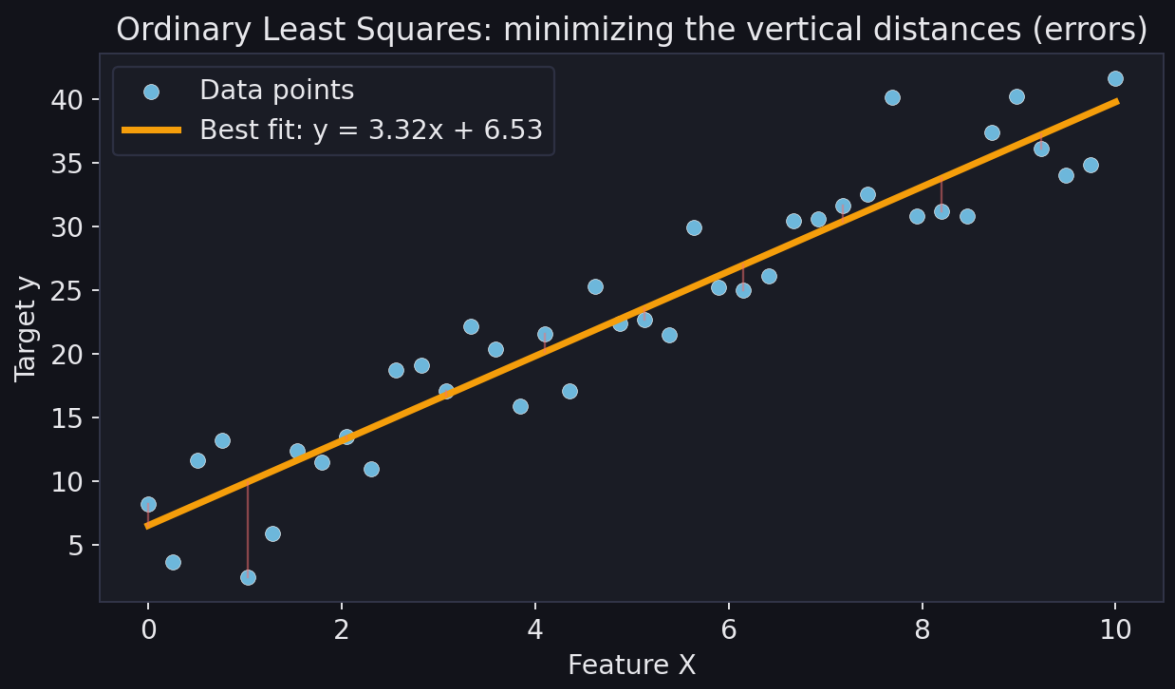


Figure 1.1 — The red segments are the errors (residuals). Linear Regression finds the exact line that makes the sum of the squared lengths of those red segments as small as possible.

1.1 — How scikit-learn Solves It: Ordinary Least Squares (OLS)

In scikit-learn, the plain LinearRegression class does not "learn" step-by-step at all. Instead, it plugs your data directly into a closed-form mathematical formula called the Normal Equation and jumps straight to the exact best-fitting line in one shot:

$$\theta = (X^T X)^{-1} X^T y$$

Here X is the matrix of all your input features, y is the vector of target values, and θ (theta) is the vector of weights (including the intercept) that comes out the other end. Because this is a direct calculation rather than trial-and-error, LinearRegression is fast, deterministic (same data always gives the same answer), and needs almost no configuration.

1.2 — The Only 4 Parameters of LinearRegression()

Because it solves the equation directly, plain LinearRegression only exposes four parameters — and only two of them actually change your model's predictions.

Parameter	Default	What it does (plain English)
fit_intercept	True	If True, the line can cross the Y-axis anywhere. If False, the line is forced through the origin (0,0). Keep True unless you know x=0 must give y=0 (e.g. 0 sq.ft = $0 price).
positive	False	Forces every coefficient (weight) to be non-negative (≥ 0). Use it when a feature can only ever help, never hurt — e.g. adding more items to a cart can't reduce total weight.
copy_X	True	Makes a safety copy of your input data before doing math on it. Setting False saves a little RAM but risks silently corrupting your original dataset — always leave it True.
n_jobs	None	How many CPU cores to use. None = 1 core, -1 = all cores. Only affects speed, never accuracy.

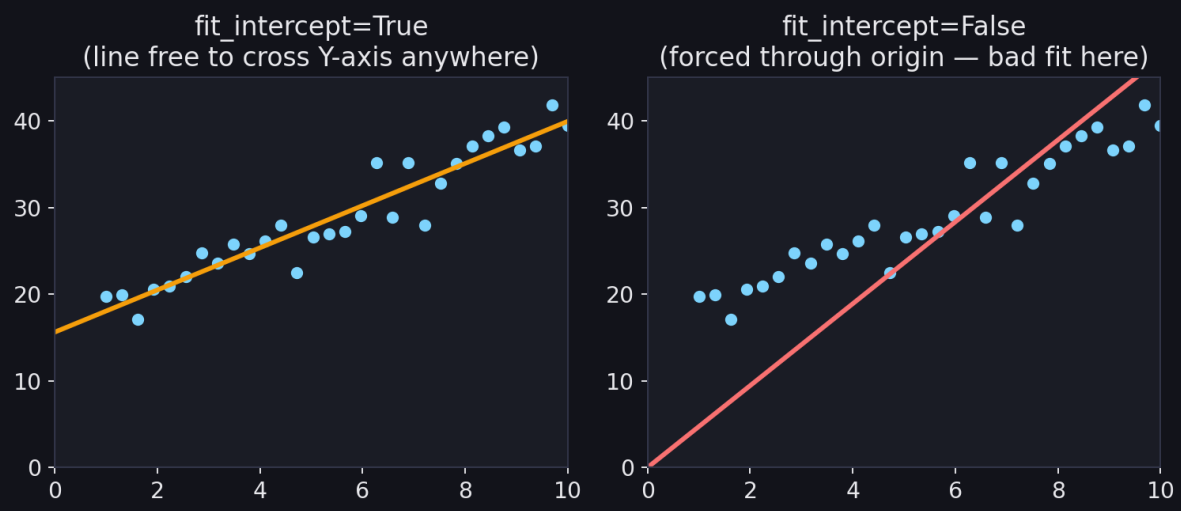


Figure 1.2 — fit_intercept=True lets the line sit wherever fits best; fit_intercept=False forces it through the origin, which can badly hurt the fit if your real data doesn't naturally pass through (0,0).

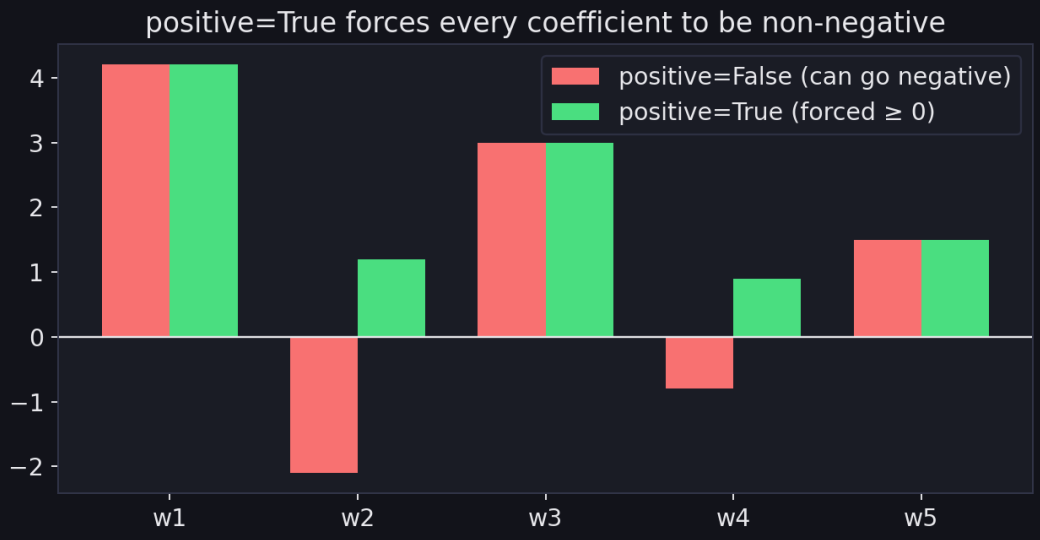


Figure 1.3 — positive=True clips every negative coefficient up to zero or above, which is useful only when negative influence is physically impossible.

NOTE

If you see a parameter called normalize in an older tutorial or course, ignore it — it was completely removed from modern scikit-learn. Use a StandardScaler inside a Pipeline instead if you need to scale your features.

1.3 — Tuning LinearRegression with GridSearchCV

GridSearchCV automatically tries every combination of parameter values you give it and tells you which combination scores best on cross-validation. You should only put parameters in the grid that can actually change accuracy — never ones that only affect memory or speed.

Parameter	Values to try	Why
fit_intercept	[True, False]	Tests whether forcing the line through (0,0) helps or hurts generalization.
positive	[True, False]	Tests whether restricting weights to positive values reduces noise.
copy_X	Do not tune	Keep default True — protects your original dataset, no accuracy effect.
n_jobs	Do not tune	Pass n_jobs=-1 directly into GridSearchCV() itself for faster tuning.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import make_regression

# 1. Generate sample data
X, y = make_regression(n_samples=500, n_features=5, noise=15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Initialize the base model
lr = LinearRegression()

# 3. Define the parameter grid (2 x 2 = 4 total combinations)
param_grid = {
    'fit_intercept': [True, False],
    'positive': [True, False]
}

# 4. Set up GridSearchCV (cv=5 means 5-fold cross-validation)
grid_search = GridSearchCV(
    estimator=lr,
    param_grid=param_grid,
    scoring='r2',      # You can also use 'neg_mean_squared_error'
    cv=5,
    n_jobs=-1          # Use all CPU cores here!
)

# 5. Fit to training data and get results
grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation R2 Score:", grid_search.best_score_)
print("Test Set R2 Score:", grid_search.score(X_test, y_test))
```

PRO TIP

Because plain LinearRegression only has 4 real combinations to try (fit_intercept × positive), tuning it almost never boosts accuracy. When people talk about "tuning linear regression" in the real world, they almost always mean tuning a regularized version instead — Ridge, Lasso, or ElasticNet — which adds penalty parameters that genuinely prevent overfitting. That's exactly where Part 3 of this guide is heading.

If you want a high-impact grid right now: swap in ElasticNet

- `alpha`: [0.001, 0.01, 0.1, 1.0, 10.0, 100.0] — overall penalty strength (higher = simpler model)
- `l1_ratio`: [0.1, 0.3, 0.5, 0.7, 0.9, 1.0] — balances Ridge (0) against Lasso (1)
- `max_iter`: [1000, 5000] — number of steps the solver takes to find the line

PART 2 · THE BRIDGE

Why Does Gradient Descent Even Exist?

If the Normal Equation already gives the exact best line in one shot, why do entire courses spend weeks teaching Gradient Descent, Momentum, RMSprop, and L1/L2 regularization on top of Linear Regression? The answer: Linear Regression is the simplest possible model, so it's the easiest place to learn the iterative, step-by-step optimization ideas that power every Neural Network. Once you understand how Gradient Descent nudges a straight line into place, you already understand 80% of how a deep neural network learns.

Two ways to reach the same straight line

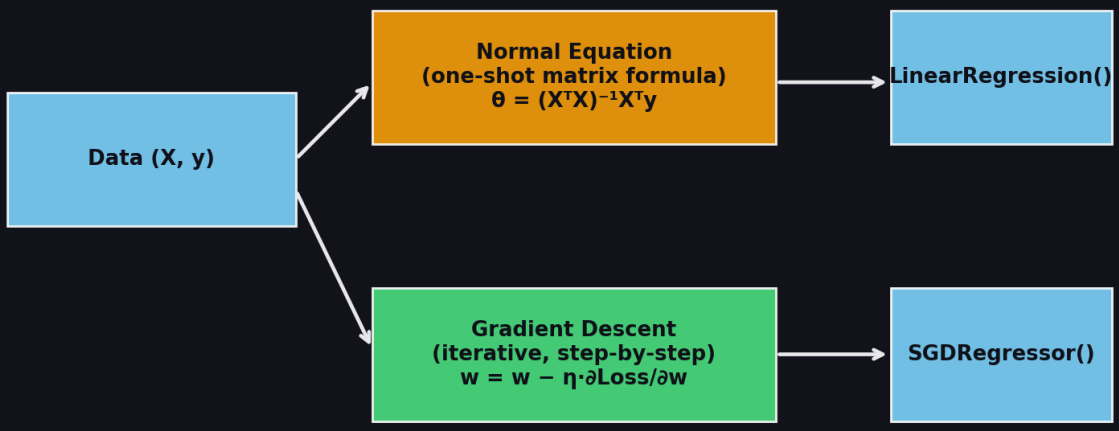


Figure 2.1 — Two completely different roads to the same straight line. LinearRegression() takes the direct mathematical shortcut. SGDRegressor() walks downhill step-by-step, which is slower per-fit but scales to huge data and unlocks regularization controls that the Normal Equation route doesn't expose in the same way.

2.1 — The Core Idea of Gradient Descent

Imagine standing on a hill of "total error" and wanting to walk downhill to the lowest point (the smallest possible error). At every step, Gradient Descent looks at the slope right under your feet (the gradient) and takes a step in the downhill direction:

$$w = w - \eta \cdot \partial Loss/\partial w$$

- w — the current weight (what we are adjusting)
- η (eta) — the learning rate: how big a step to take
- $\partial Loss/\partial w$ — the gradient: the slope of the error at the current point

Repeat that update thousands of times, and the weight slowly rolls downhill into the valley — the point of minimum error. That's it. That single idea, repeated with small variations, is the engine behind linear models, logistic regression, and every deep neural network in production today.

PART 3 · INTERMEDIATE

SGDRegressor: Gradient Descent in scikit-learn

If you want iterative Gradient Descent with L1/L2 regularization inside scikit-learn, you switch from LinearRegression to SGDRegressor (Stochastic Gradient Descent Regressor). Think of it like teaching a beginner to draw a line through a scatter plot one dot at a time — it nudges the line after looking at each row of data.

> **WARNING**
> Gradient Descent is very sensitive to feature scale. Always scale your data with StandardScaler first — otherwise the gradients can explode into huge, unstable numbers.

3.1 — Careful: scikit-learn Flips the Textbook Names!

In theory classes, Gradient Descent uses α (alpha) for the learning rate and λ (lambda) for regularization strength. In SGDRegressor, it's the opposite of what you might guess:

Textbook symbol	Theory meaning	SGDRegressor parameter
η (eta)	Learning rate (step size)	`eta0`
λ (lambda)	Regularization strength	`alpha`

3.2 — Full Parameter Reference

penalty — which regularizer to apply (default: 'l2')

- 'l2' (Ridge) — like a rubber band on every weight: pulls large weights toward zero but never all the way to exactly 0.
- 'l1' (Lasso) — a strict filter: pushes unhelpful/noisy feature weights all the way to exact 0, effectively deleting that feature.
- 'elasticnet' — combines both: deletes useless features (like L1) while shrinking the remaining useful ones (like L2).
- None — turns regularization off completely (pure textbook Gradient Descent).

Values for GridSearchCV: `['l2', 'l1', 'elasticnet', None]` — you rarely know in advance whether your dataset has useless features to delete (l1) or all useful features that just need

shrinking (l2), so let the data decide.

alpha — regularization strength, i.e. λ (default: 0.0001)

Too small (e.g. 0.000001) and the punishment is barely noticeable — the model may overfit. Too large (e.g. 100) and the punishment crushes every weight to zero — a flat, useless line (underfitting). Always tune alpha on a logarithmic scale (×10 each step) since small linear jumps make almost no difference.

Values for GridSearchCV: `[0.0001, 0.001, 0.01, 0.1, 1.0]`

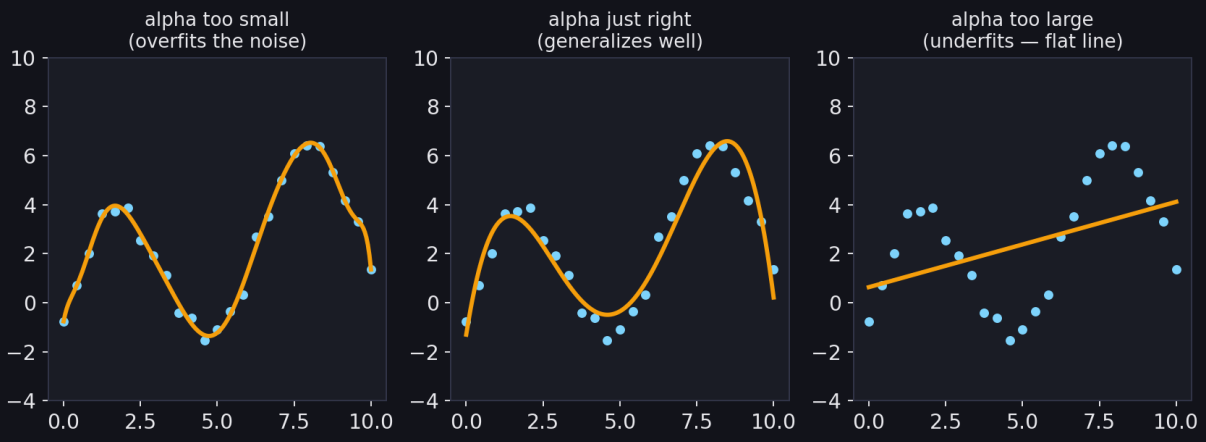


Figure 3.1 — Too little regularization chases every noisy wiggle in the data (overfitting); too much flattens the model into a nearly straight, uninformative line (underfitting). The right alpha sits in between.

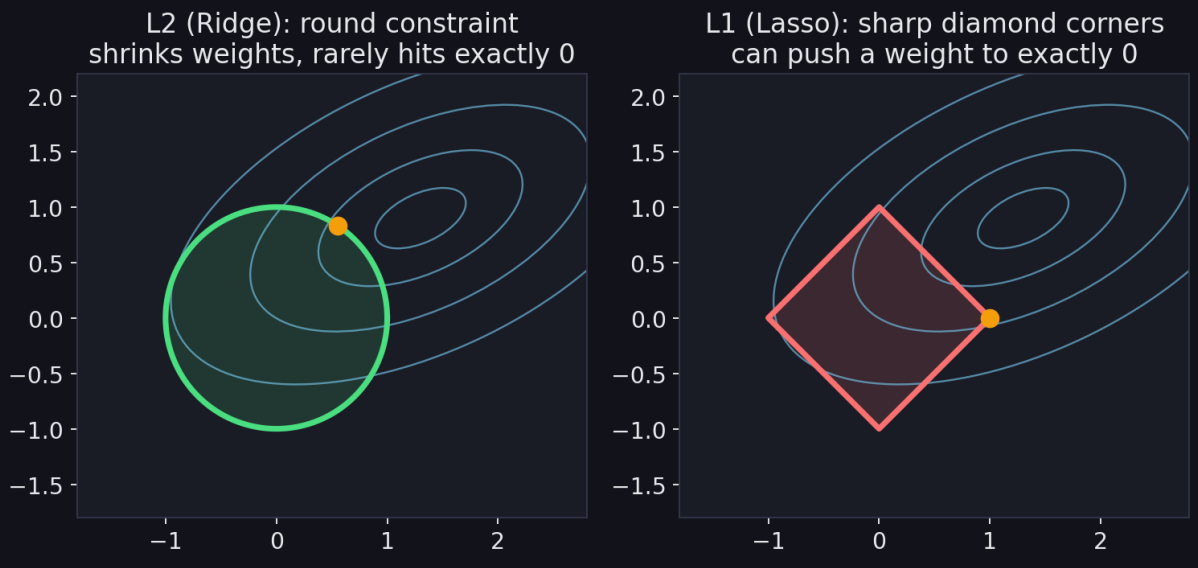


Figure 3.2 — Geometrically, L2's round constraint region (left) tends to shrink weights smoothly toward — but not exactly to — zero. L1's diamond-shaped region (right) has sharp corners that the loss contours often land on exactly, which is why L1 can zero out a weight completely.

l1_ratio — the L1/L2 mixing slider (only used when penalty='elasticnet')

An l1_ratio of 0.7 means the penalty is 70% L1 and 30% L2. Default: 0.15.

Values for GridSearchCV: `[0.15, 0.5, 0.85]` — 0.15 tests mostly L2, 0.5 an even split, 0.85 mostly L1 (0 and 1 aren't needed since penalty already covers pure 'l2' and pure 'l1').

eta0 & learning_rate — controlling the step size

`eta0` (default 0.01) is your starting learning rate. Too big (e.g. 1.0) and you leap right over the valley, bouncing until the math crashes (NaN). Too small (e.g. 0.00001) and training finishes before you ever reach bottom.

Values for GridSearchCV: `[0.001, 0.01, 0.1]`

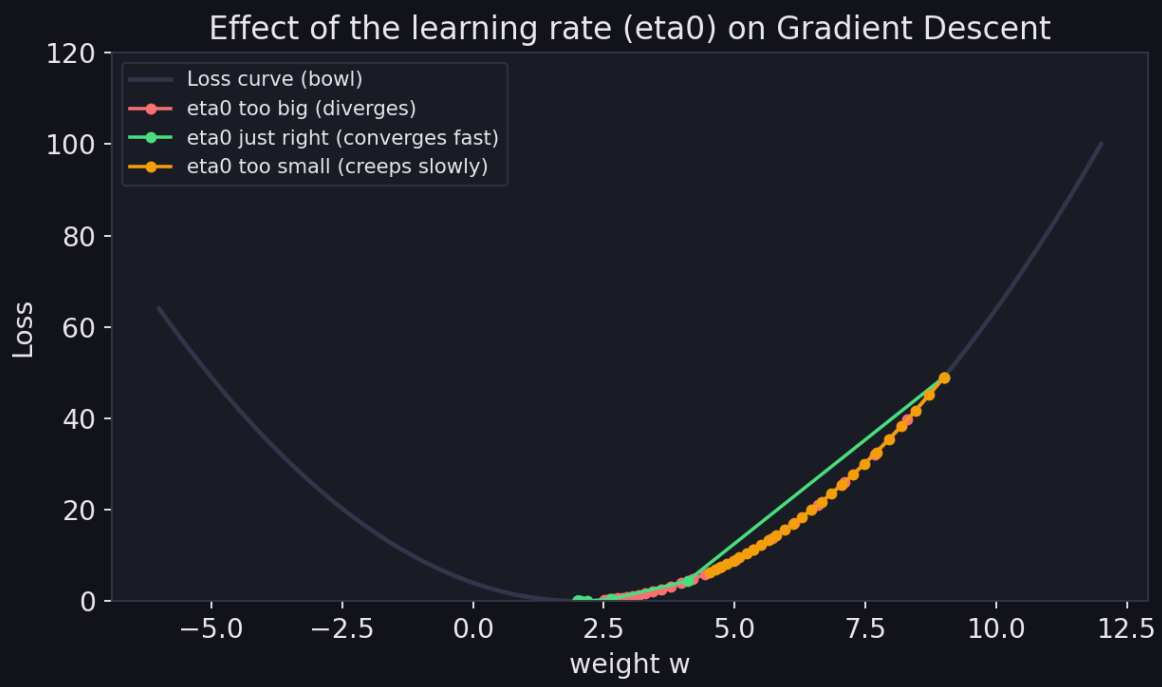


Figure 3.3 — A learning rate that's too big overshoots and diverges; one that's too small crawls toward the minimum; the right one converges quickly and smoothly.

`learning_rate` controls how `eta0` changes over time (default: `'invscaling'`):

- 'constant' — never changes; every step uses the exact same eta0.
- 'invscaling' — shrinks step size after every update via $\eta = \text{eta0} / t^{\text{power_t}}$. Can shrink too early.
- 'adaptive' (usually best) — keeps eta0 constant while the error keeps improving; the moment it plateaus, the step size is divided by 5, and this repeats until the step is tiny.

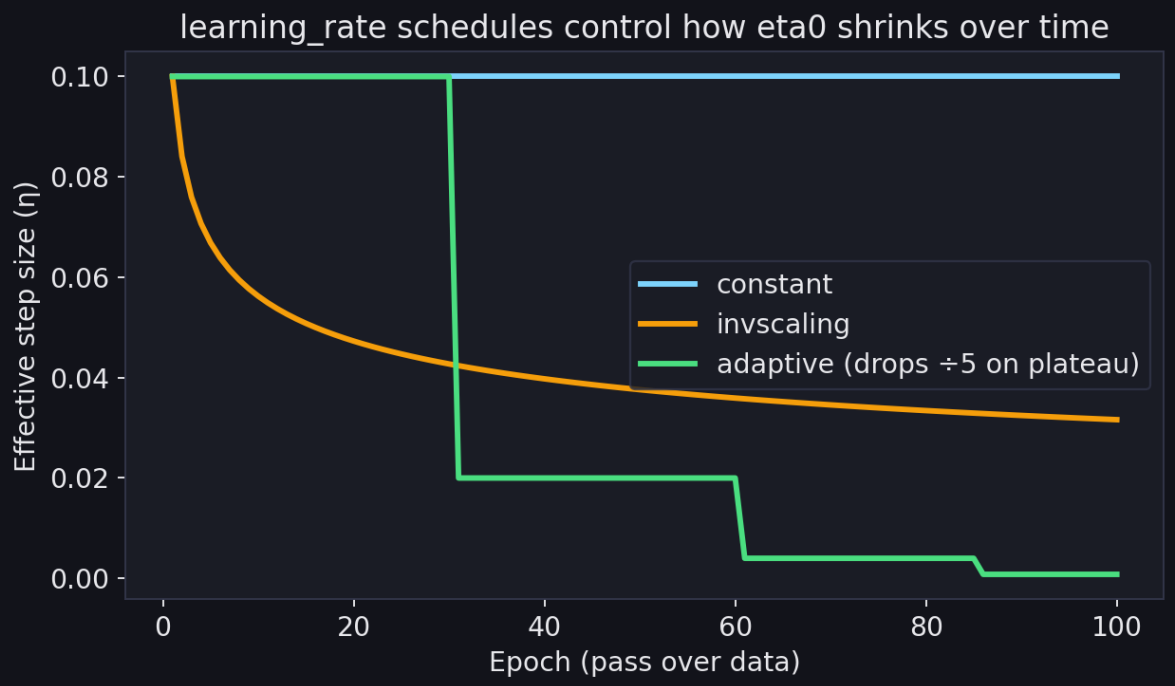


Figure 3.4 — How the effective step size changes across training for each schedule. 'adaptive' rides at full speed, then backs off in stages exactly when progress stalls.

loss — the error formula being minimized (default: 'squared_error')

- 'squared_error' — standard Mean Squared Error. Squaring means one extreme outlier gets squared into a huge penalty and can pull the whole line toward it.
- 'huber' — behaves like squared error for normal points but switches to linear (absolute) error for extreme outliers, so outliers can't dominate the fit.

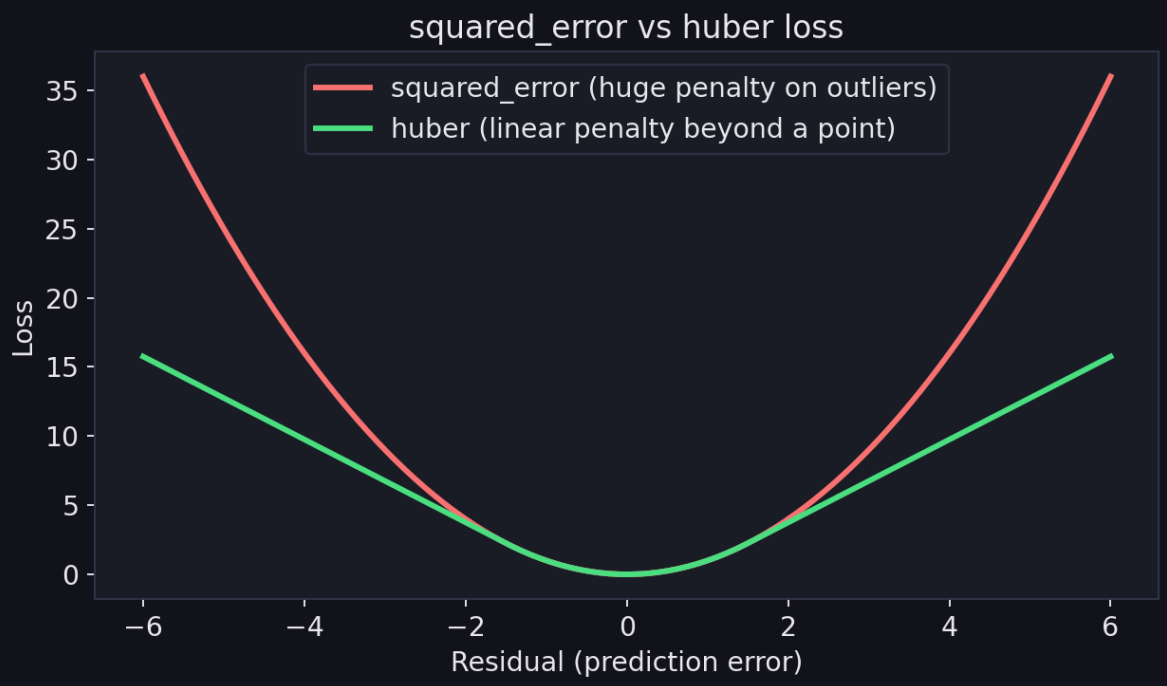


Figure 3.5 — squared_error's penalty explodes quadratically for large residuals; huber flattens into a straight line beyond a threshold, which is why it's more robust to outliers.

max_iter & tol — when to stop training

`max_iter` (default 1000) is the maximum number of passes (epochs) over your data. `tol` (default 0.001) is the "good enough" rule: if the error improves by less than tol for 5 epochs in a row, training stops early. Smaller learning rates need more epochs to finish, so set `max_iter` to `[2000, 5000]` to avoid an annoying ConvergenceWarning.

3.3 — Complete SGDRegressor Cheat-Sheet Grid

```
param_grid = {
    # 1. Error Formula
    'sgd__loss': ['squared_error', 'huber'],

    # 2. Regularization (L1 / L2)
    'sgd__penalty': ['l2', 'l1', 'elasticnet', None],
    'sgd__alpha': [0.0001, 0.001, 0.01, 0.1, 1.0],
    'sgd__l1_ratio': [0.15, 0.5, 0.85],

    # 3. Gradient Descent Step Size
    'sgd__learning_rate': ['constant', 'invscaling', 'adaptive'],
    'sgd__eta0': [0.001, 0.01, 0.1],

    # 4. Epochs (give it enough steps to finish!)
    'sgd__max_iter': [3000]
}
```

3.4 — Full Working Example

```
from sklearn.linear_model import SGDRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import make_regression

# 1. Data setup
X, y = make_regression(n_samples=500, n_features=10, noise=15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Pipeline (Scaling is mandatory for Gradient Descent!)
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('sgd', SGDRegressor(random_state=42))
])

# 3. Parameter grid (use 'sgd__' prefix for pipeline steps)
param_grid = {
    'sgd__penalty': ['l2', 'l1', 'elasticnet'],
    'sgd__alpha': [0.0001, 0.001, 0.01, 0.1],
    'sgd__l1_ratio': [0.15, 0.5, 0.85],
    'sgd__learning_rate': ['constant', 'adaptive'],
    'sgd__eta0': [0.001, 0.01, 0.1],
    'sgd__max_iter': [1000]
}

grid = GridSearchCV(pipe, param_grid, cv=5, scoring='r2', n_jobs=-1)
grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Test R2 Score:", grid.score(X_test, y_test))
```

PART 4 · ADVANCED

Momentum & RMSprop: Building Optimizers From Scratch

scikit-learn's SGDRegressor does not include Momentum or RMSprop, because those optimizers were designed for deep learning frameworks like PyTorch and TensorFlow. To actually compare Regular Gradient Descent vs. Momentum vs. RMSprop — with L1/L2 regularization — inside scikit-learn's GridSearchCV, you wrap the exact theory math into a custom estimator.

4.1 — The Three Update Rules

- Regular GD: $w = w - \eta \cdot dw$
- Momentum: $v = \beta v + (1-\beta)dw \rightarrow w = w - \eta \cdot v$
- RMSprop: $s = \beta s + (1-\beta)dw^2 \rightarrow w = w - (\eta/\sqrt{(s+\varepsilon)}) \cdot dw$

Momentum keeps a running, decaying average of past gradients (like a ball rolling downhill that builds up speed), which helps it power through small bumps but can cause it to overshoot the minimum. RMSprop keeps a running average of the squared gradients and divides the step size by its square root, which automatically shrinks the step in steep directions and keeps it larger in flat ones.

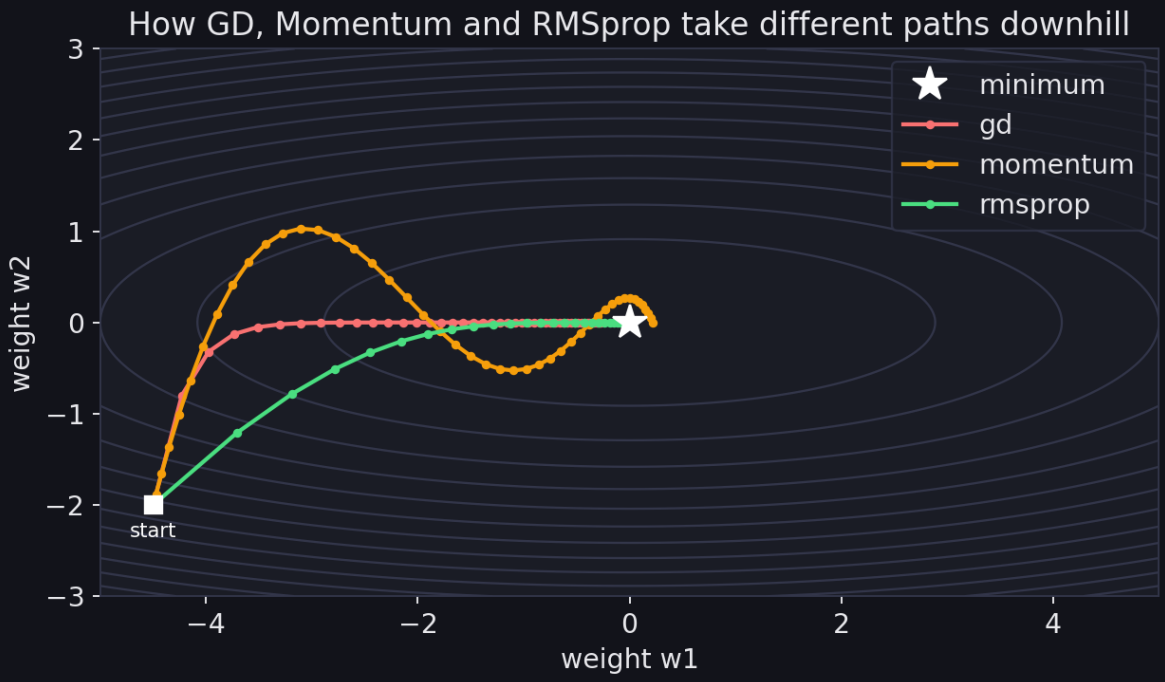


Figure 4.1 — On an elongated bowl-shaped loss surface, plain GD converges slowly along the flat direction, Momentum overshoots past the minimum before settling, and RMSprop rescales each direction automatically for a smoother, more direct path.

4.2 — Complete Custom Estimator Code

```
import numpy as np
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import make_regression

class CustomGDLinearRegression(BaseEstimator, RegressorMixin):
    def __init__(self, optimizer='gd', lr=0.01, epochs=500, beta=0.9, l1=0.0, l2=0.0):
        self.optimizer = optimizer  # 'gd', 'momentum', or 'rmsprop'
        self.lr = lr                # Learning rate
        self.epochs = epochs        # Number of iterations
        self.beta = beta            # Decay rate for Momentum / RMSprop
        self.l1 = l1                # L1 penalty (Lasso)
        self.l2 = l2                # L2 penalty (Ridge)
```

```
    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w_ = np.zeros(n_features)
        self.b_ = 0.0

        vw, vb = np.zeros(n_features), 0.0   # Momentum buffers
        sw, sb = np.zeros(n_features), 0.0   # RMSprop buffers
        eps = 1e-8

        for _ in range(self.epochs):
            y_pred = np.dot(X, self.w_) + self.b_
            error = y_pred - y

            dw = (2 / n_samples) * np.dot(X.T, error) + self.l1 * np.sign(self.w_) + 2 *
                self.l2 * self.w_
            db = (2 / n_samples) * np.sum(error)
```

```
            if self.optimizer == 'gd':
                self.w_ -= self.lr * dw
                self.b_ -= self.lr * db

            elif self.optimizer == 'momentum':
                vw = self.beta * vw + (1 - self.beta) * dw
                vb = self.beta * vb + (1 - self.beta) * db
                self.w_ -= self.lr * vw
                self.b_ -= self.lr * vb

            elif self.optimizer == 'rmsprop':
                sw = self.beta * sw + (1 - self.beta) * (dw ** 2)
                sb = self.beta * sb + (1 - self.beta) * (db ** 2)
                self.w_ -= (self.lr / (np.sqrt(sw) + eps)) * dw
                self.b_ -= (self.lr / (np.sqrt(sb) + eps)) * db

        return self

    def predict(self, X):
        return np.dot(X, self.w_) + self.b_
```

```
# Data & Pipeline
X, y = make_regression(n_samples=500, n_features=10, noise=15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', CustomGDLinearRegression())
])

param_grid = {
    'model__optimizer': ['gd', 'momentum', 'rmsprop'],
    'model__lr': [0.001, 0.01, 0.05],
    'model__beta': [0.9, 0.99],
    'model__l1': [0.0, 0.01, 0.1],
    'model__l2': [0.0, 0.01, 0.1],
    'model__epochs': [500]
}

grid = GridSearchCV(pipe, param_grid, cv=5, scoring='r2', n_jobs=-1)
grid.fit(X_train, y_train)

print("Winning Optimizer & Params:", grid.best_params_)
print("Test R2 Score:", grid.score(X_test, y_test))
```

4.3 — Choosing Grid Values for the Custom Optimizer

Parameter	Values to try	Why
optimizer	['gd', 'momentum', 'rmsprop']	Lets GridSearchCV directly pit the three algorithms against each other.
beta	[0.9, 0.99]	0.9 is the gold-standard default for both Momentum and RMSprop in the literature.

Parameter	Values to try	Why
lr	[0.001, 0.01, 0.05]	rmsprop usually prefers smaller rates (0.001–0.01); gd/momentum often prefer 0.01–0.05.
l1, l2	[0.0, 0.01, 0.1]	Including 0.0 lets the grid test whether turning regularization off entirely wins.

PART 5 · PRO LEVEL

6 Pro Tips Every Beginner Needs

SGDRegressor updates its line after looking at one data point at a time, so it can easily waste time or get confused without a few guardrails. Here are the six most important fixes, in plain English.

Tip 1 — Don't Make GridSearchCV Repeat the Same Work

`l1_ratio` only matters when `penalty='elasticnet'`. If you dump all three parameters into a single grid dictionary, GridSearchCV will train the exact same 'l2' or 'l1' model three separate times — once per l1_ratio value — wasting time on identical results.

Fix: split your grid into two separate dictionaries: one for 'l1'/'l2' (no l1_ratio), and one for 'elasticnet' (with l1_ratio).

```
param_grid = [
    # Box 1: Test L1 and L2 (No l1_ratio here!)
    {'sgd__penalty': ['l1', 'l2'], 'sgd__alpha': [0.001, 0.01, 0.1]},

    # Box 2: Test ElasticNet (Put l1_ratio ONLY here!)
    {'sgd__penalty': ['elasticnet'], 'sgd__alpha': [0.001, 0.01, 0.1], 'sgd__l1_ratio': [0.15,
        0.5, 0.85]}
]
```

Tip 2 — Use average=True to Avoid a Bad Last Dot

Because SGDRegressor moves the line after every single row, an extreme outlier appearing right at the very end of training can yank the final line off course.

Fix: set `SGDRegressor(average=True)`. Instead of keeping only the very last line drawn, the model averages all the lines it drew during training, smoothing out the wobble.

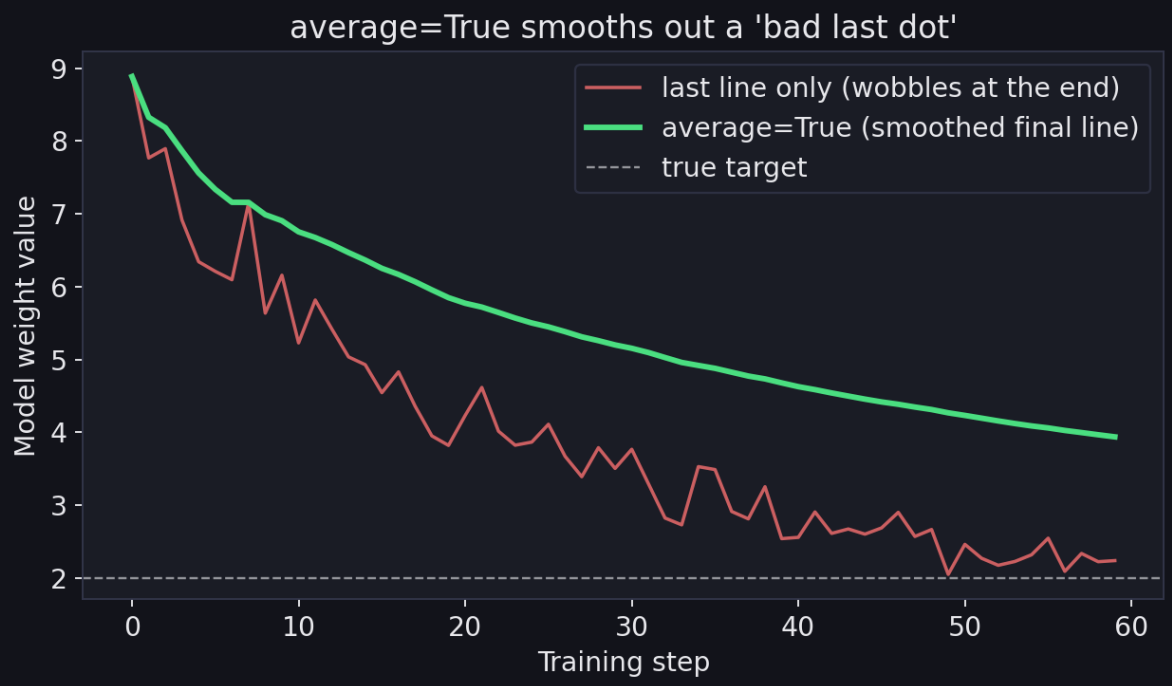


Figure 5.1 — The un-averaged line keeps wobbling right up to the last steps. average=True smooths that noise into a stable final estimate.

Tip 3 — Why Your Score Suddenly Becomes -999999999999.0

If your target column has huge numbers (e.g. house prices around $10,000,000), the very first prediction starts near $0. The resulting error is enormous, so Gradient Descent panics and takes a massive leap to correct it — overshooting into billions, then trillions, until the math explodes into NaN.

Fix: if y has huge numbers, rescale it first (e.g. measure prices in millions like 10.5 instead of 10,500,000), or use a very small learning rate such as `eta0 = 0.0001`.

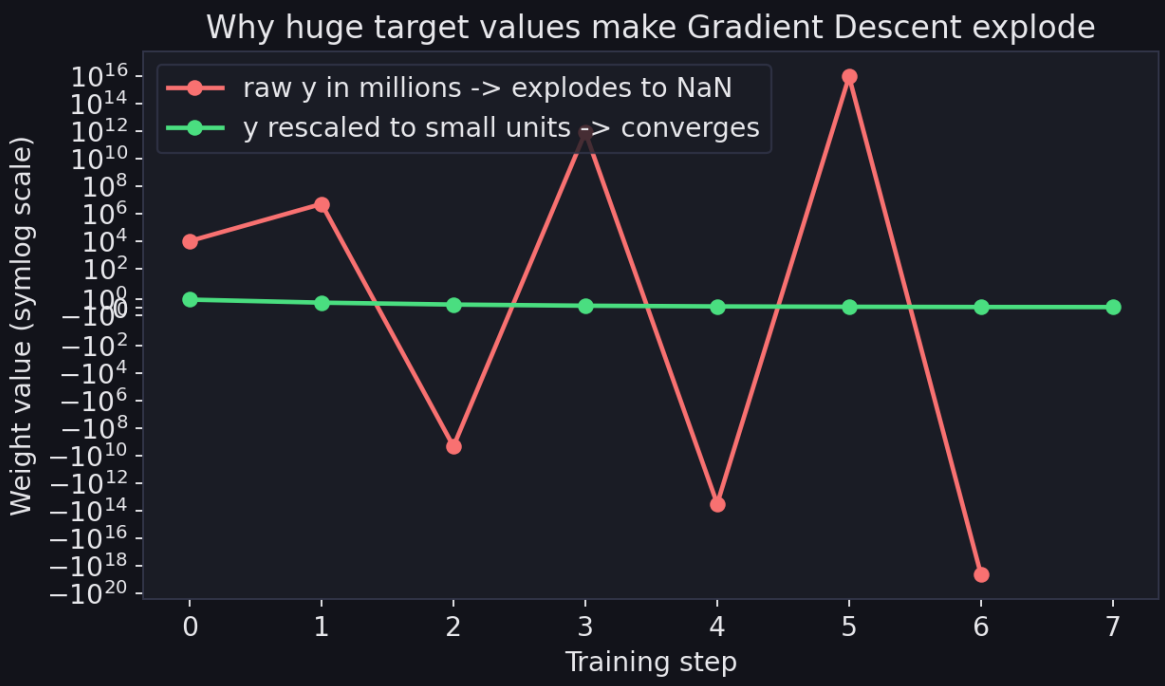


Figure 5.2 — Unscaled multi-million-scale targets can send weight values exploding to infinity within a handful of steps; rescaling the target keeps training numerically stable.

Tip 4 — Use early_stopping=True (Don't Watch a Bad Movie to the End)

When GridSearchCV tests 50 different combinations, many will be terrible. With `max_iter=3000`, Python will stubbornly force a terrible model through all 3,000 loops even if it stopped learning on loop #5.

Fix: add `early_stopping=True` — this checks the score after every loop and stops immediately once it hasn't improved for 5 loops in a row, moving on to the next setting.

Tip 5 — How to See Your Final Equation (y = wx + b)

If you used a Pipeline (which bundles StandardScaler and SGDRegressor together like a gift box inside a shipping box), calling `grid.coef_` directly fails, because the outer Pipeline object doesn't have a coef_ attribute of its own.

Fix: open the outer box (best_estimator_) and grab the named step inside it:

```
print("Weights (w):", grid.best_estimator_.named_steps['sgd'].coef_)
print("Intercept (b):", grid.best_estimator_.named_steps['sgd'].intercept_)
```

Tip 6 — Big Data vs. Small Data: Cargo Truck vs. Sports Car

Because courses teach Gradient Descent first, it's tempting to use SGDRegressor for everything. But since it learns one row at a time with random shuffling, it's noisier on small datasets.

Situation	Recommended tool
Practicing Gradient Descent, or 100,000+ rows too big for RAM at once	SGDRegressor
A normal-sized table (100 – 50,000 rows), e.g. a Kaggle competition	ElasticNet / Ridge / Lasso

The Ultimate Beginner Template

This single template combines every fix from Part 5: StandardScaler, a smart split grid for L1/L2/ElasticNet, early stopping, and weight averaging.

```
# ======================================================================
# ULTIMATE BEGINNER TEMPLATE: Linear Regression with Gradient Descent
# Includes: StandardScaler, L1/L2/ElasticNet, Early Stopping & Averaging
# ======================================================================

from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDRegressor
from sklearn.pipeline import Pipeline

# STEP 1: Load Your Data
# (Replace make_regression with your own X and y from pandas!)
# Pro-Tip #3: If y has huge numbers (millions), divide it first!
X, y = make_regression(n_samples=1000, n_features=8, noise=15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# STEP 2: Build the Pipeline (Scaler + Gradient Descent Model)
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('sgd', SGDRegressor(
        max_iter=3000,
        early_stopping=True,      # Pro-Tip #4
        n_iter_no_change=5,
        random_state=42
    ))
])
```

```
# STEP 3: Smart Parameter Grid (Pro-Tip #1 & #2)
param_grid = [
    {   # BOX 1: Pure L1 (Lasso), Pure L2 (Ridge), or No Penalty
        'sgd__loss': ['squared_error', 'huber'],
        'sgd__penalty': ['l2', 'l1', None],
        'sgd__alpha': [0.0001, 0.001, 0.01, 0.1, 1.0],
        'sgd__learning_rate': ['adaptive', 'invscaling'],
        'sgd__eta0': [0.001, 0.01, 0.1],
        'sgd__average': [True, False]
    },
    {   # BOX 2: ElasticNet (Combines L1 + L2 using the l1_ratio slider)
        'sgd__loss': ['squared_error', 'huber'],
        'sgd__penalty': ['elasticnet'],
        'sgd__alpha': [0.0001, 0.001, 0.01, 0.1, 1.0],
        'sgd__l1_ratio': [0.15, 0.5, 0.85],
        'sgd__learning_rate': ['adaptive', 'invscaling'],
        'sgd__eta0': [0.001, 0.01, 0.1],
        'sgd__average': [True, False]
    }
]
```

```
# STEP 4: Run GridSearchCV (5-Fold Cross-Validation)
grid = GridSearchCV(estimator=pipe, param_grid=param_grid, cv=5, scoring='r2', n_jobs=-1)
print("Training and tuning models... Please wait.")
grid.fit(X_train, y_train)

# STEP 5: Check Results & Accuracy
print("\n--- RESULTS ---")
print("Best Settings Found:", grid.best_params_)
print(f"Training CV R2 Score: {grid.best_score_:.4f}")
print(f"Final Test R2 Score:  {grid.score(X_test, y_test):.4f}")

# STEP 6: Extract Final Equation (y = wx + b) (Pro-Tip #5)
best_sgd_model = grid.best_estimator_.named_steps['sgd']
print("\n--- FINAL EQUATION NUMBERS ---")
print("Feature Weights (w):", best_sgd_model.coef_)
print("Intercept (b):      ", best_sgd_model.intercept_[0])

# predictions = grid.predict(X_test)
```

Using This on Your Own CSV File

When you move from practice data to a real project using pandas, you only need to change Step 1:

```
import pandas as pd

df = pd.read_csv("your_dataset.csv")
X = df.drop(columns=["target_column_name"])  # All columns except what you're predicting
y = df["target_column_name"]                 # The column you want to predict
```

PART 7 · REFERENCE

Quick Reference Cheat Sheets

A. Which Model Should I Use?

If you want...	Use this
The fastest, exact fit on a normal-sized dataset	LinearRegression()
A physically-forced non-negative relationship	LinearRegression(positive=True)
To shrink weights smoothly, keep all features	Ridge / SGDRegressor(penalty='l2')
To automatically delete useless features	Lasso / SGDRegressor(penalty='l1')
A balance of both L1 and L2	ElasticNet / SGDRegressor(penalty='elasticnet')
To learn on huge / streaming data that won't fit in RAM	SGDRegressor
To experiment with Momentum / RMSprop theory	Custom BaseEstimator (Part 4)

B. Master Parameter Glossary

Term	Meaning
w (weight / coefficient)	How much the prediction changes per unit of a feature
b (bias / intercept)	The prediction when all features are 0
eta0	Starting learning rate — the size of each Gradient Descent step
alpha (in SGDRegressor)	Regularization strength, i.e. theory's lambda (λ) — NOT the learning rate
l1_ratio	Mix between L1 and L2 penalties, only active when penalty='elasticnet'
beta	Decay rate controlling how much of the past gradient Momentum/RMSprop remembers
epoch	One full pass of the training algorithm over the entire dataset
tol	Minimum improvement required to keep training instead of stopping early

Term	Meaning
R2 score	Fraction of variance in y explained by the model (1.0 = perfect, 0 = no better than the mean)

C. The Big Picture

Every model in this guide answers the exact same question — what are the best values of w and b? — using progressively more powerful and flexible tools:

- LinearRegression — exact formula, no regularization, no tuning needed
- Ridge / Lasso / ElasticNet — exact-ish formula + a regularization penalty to fight overfitting
- SGDRegressor — the same penalties, but learned iteratively so it scales to huge data
- Custom Momentum / RMSprop estimator — the same iterative idea, but with smarter, adaptive step sizes — the same core trick used inside modern neural network training

---

End of tutorial — built from your notes on LinearRegression, SGDRegressor, GridSearchCV, and custom Gradient-Descent optimizers.